report

Photal Khan

8 September 2025

1 Introduction

Mobile applications are now very important to everyday life, but even small bugs can damage mobile application and user trust. One of the most common weakness is unusual text inputs users type everything from names and messages to sql queries, and these unexpected inputs easily cause mobile application to crash the application. Traditional testing methods commonly explore these unusual cases, leaving many vulnerabilities undiscovered. This paper is using InputBlaster, a system that uses chantgpt to generate different and unusual text inputs. By targeting the inputs fields most likely to break mobile application , InputBlaster is used to help developers undiscover or uncover hidden bugs and improve the reliability of mobile applications.

2 how mutation rules generated according to paper

In input blaster paper mutation rules are not written manually instead they are automatically generated by LLM (chat-gpt) The LLM first generates valid imputes for field like username at the same time its also infers constraints behind this input (password must be characters or should be contains number , special characters and alphabet) InputBlaster parses the view hierarchy file of the app's GUI (collected using tools like UIAutomator). From this file, it extracts: Hint text (e.g., "Enter Username"). Resource IDs (developer-given IDs). Existing text values.

- **Widget context** (e.g., label "Enter username").
- **Nearby widgets** (e.g., "Height", "Weight").
- **Dynamic hints** (error messages shown when user enters invalid text).
- **Constraint categories** (from their pilot study: intra-widget and inter-widget constraints).

Example

- **Context**: “Enter Username” field on Sign Up page
- **Constraint**: Must be 5–15 characters, only alphanumeric + underscore allowed, no spaces.
- **Dynamic Hint**: “Username must be at least 5 characters” OR “Special characters not allowed.”constraints).

3 How Constraints Are Extracted in InputBlaster

The constraints are extracted in three ways

3.1 Context Extraction (Static Information)

- InputBlaster parses the view hierarchy file of the app’s GUI with UIAutomator.
- from view hierarchy it extracts
  - hint text (username)
  - resource ids text (ids of widgets)
  - existing text value
  - its also collect local context from near widgets(like password box near to username box)
  - this helps to chat-gpt or any llm to understand what kind of input is valid

3.2 Dynamic Hint Extraction

- InputBlaster parses the view hierarchy file of the app’s GUI with UIAutomator.
- when user inter value app should show error on wrong value(dynamic hint)
  - Enter ”abc” into password hows: “Password must contain at least 1 number.

3.3 Candidate Constraint Categories (From Pilot Study)

- The authors manually studied thousands of apps and summarized constraint categories:
- when user inter value app should show error on wrong value(dynamic hint)
- Intra-widget constraints (single field)
- Explicit

  - only numbers allowed," "must be $\geq$ 5 characters.

- Implicit
  - Password must contain uppercase letters.

- Inter-widget constraints (multiple fields)
  - Relations across fields, e.g., "Confirm password must match password," "Departure date ¡ Return date."

4 Test Case Generation in InputBlaster

4.1 1. Valid Input & Constraint Inference

The chatgpt first create valid sets of inputs passes to the widget (e.g. $\texttt{abbas}_1 23@foremailorusernameandsamet$ $15characters)theseconstraintsarelaterusedformutateinput$

4.2 2. Mutation Rule Creation

`Based on the valid input and constraints, the Chatgpt is asked to :`
`''Now generate a mutation rule that violates these constraints for input.''`

`Example of mutation rules:`

- `Insert special characters ("john$doe")`
- `Make it too short ("a")`
- `Use Unicode or emoji ("user")`
- `Enter SQL injection-like input ("admin';--")`

4.3 3. Test Generator Creation

`The Chatgpt write a test generator (like small code snippet for test generation) that will be used for mutation rules Example (Python-style generator):`

```
def username_generator():
    base = "john_doe123"
    mutations = [
        "a",                      # too short
        "superlongusername_exceedinglimit",  # too long
        "john$doe",               # special character
        "user",                 # emoji
        "admin';--",              # SQL injection
        ""
    ]
    return mutations
```

Each element in mutations[] is a test case.

4.4 4. Batch of Test Cases Produced

As like direct input generation the test generator will produces a batch of tedt cases at once (batch means whole collection of test cases here)

Example batch for username:

- "a"
- "superlongusername_exceedinglimit"
- "john$doe"
- "user"
- "admin';--"
- "" (empty input)

4.5 5. Test Case Execution

The Generated unsual inputs are automatically entered in app inputs with automator python package After input enter , the app is monitored for:

- Crash / Exception → bug detected.
- No crash → feedback loop (tells LLM to try new mutation rules next round).

4.6 6. Feedback & Iteration

- The crashing inputs will be saved when confirmed buggy test cases.
- Non-crashing inputs are used to encourage the chatgpt to explore new mutation rules.

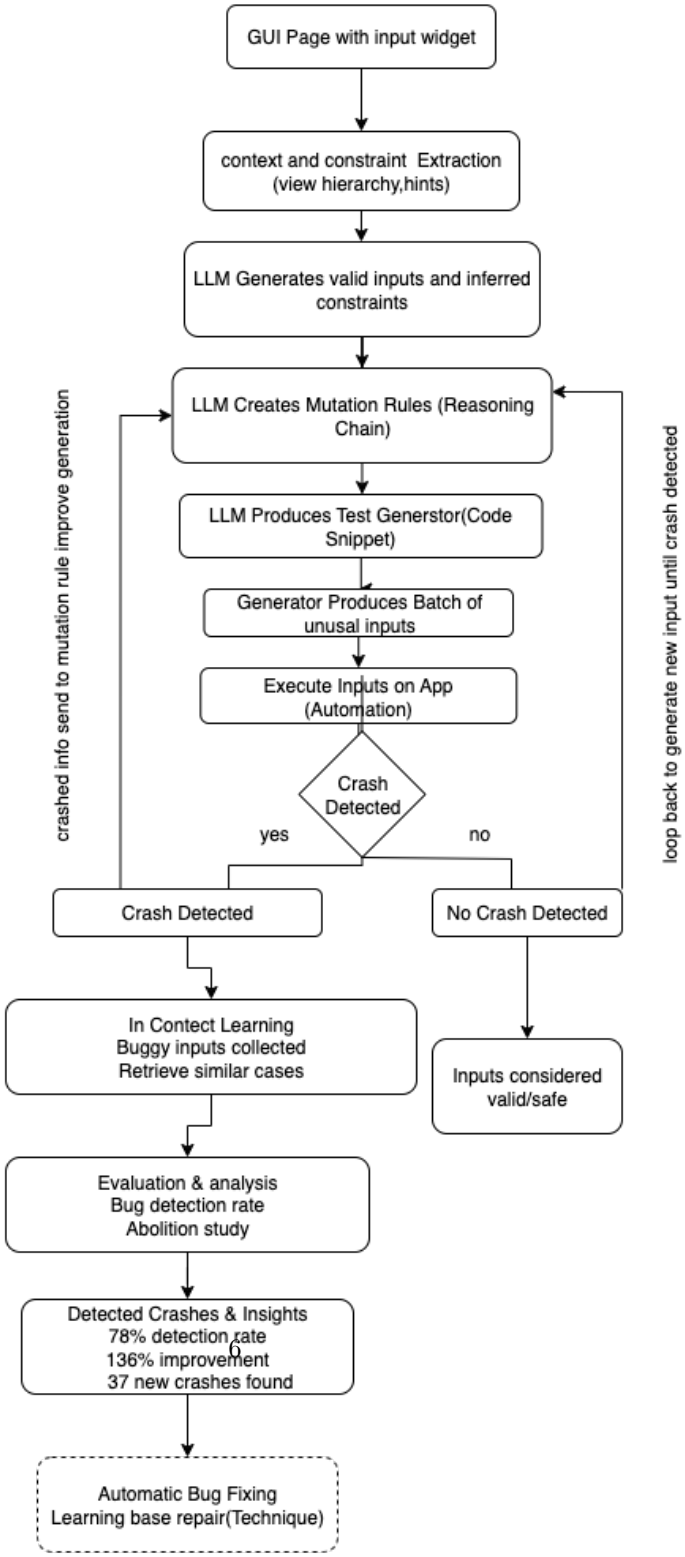
This will creates a loop for feedback to making test case generation more diverse and targeted over time.

Some Insights from this Paper

- Why not generate test cases directly? Because asking the Chatgpt for each unusual inputs are slow and costly. Instead, by generating mutation rules + code generators, InputBlaster can quickly produce hundreds of test cases.

- In-context Learning Boost: InputBlaster feeds the Chathpt with examples of past buggy inputs from github reports This helps the Chatgpt generate better mutation rules, and thus better test cases.
- Evolutionary Nature:

  The Test cases are not static. They evolve across iterations depending on crash feedback.


GUI Page with input widget
context and constraint Extraction
(view hierarchy,hints)
LLM Generates valid inputs and inferred constraints
LLM Creates Mutation Rules (Reasoning Chain)
LLM Produces Test Generstor(Code Snippet)
Generator Produces Batch of unusal inputs
Execute Inputs on App (Automation)
Crash Detected
yes
no
crashed info send to mutation rule improve generation
loop back to generate new input until crash detected
Crash Detected
No Crash Detected
In Contect Learning
Buggy inputs collected
Retrieve similar cases
Inputs considered valid/safe
Evaluation & analysis
Bug detection rate
Abolition study
Detected Crashes & Insights
78% detection rate
136% improvement
37 new crashes found
Automatic Bug Fixing
Learning base repair(Technique)

5 Literature Review

5.1 Literature Review on Mobile Application Testing

The rapid growth of smartphones and mobile applications has made application testing challenges to ensuring in quality, performances and customer/user satisfaction . Unlike traditional software, mobile applications operate across huge number of devices, platforms, networks. The paper *Mobile Application Testing: A Tutorial* by Gao, Bai, Tsai, and Uehara (2014) provides one of the foundational overviews mobile application testing fields highlights testing requirements and existing approaches.

Key Contributions

This paper is a tutorial to distinguishes mobile app testing from conventional software testing by highlighting the requirements such as mobility, usability, connectivity, and security. In this paper authors describe various testing goals, including functionality, quality of service (QoS), interoperability, and privacy. Importantly, they divide mobile testing into four infrastructures: emulation-based, device-based, cloud-based, and crowd-based testing. Each has distinct uses, advantages and limitations emulation based test case offers low cost but limited realism, device-based testing ensures accuracy but is expensive, cloud-based testing provides scalability, and crowd-based testing captures real-world diversity but is less controlled.

Related Work

The paper explained its discussion within prior research across multiple dimensions:

- White-box testing (e.g Java PathFinder) for verification of code and coverage.
- Black-box and GUI testing techniques like automated event sequence validation (AndroidRipper).
- Usability testing, usability testing are often conducted in both lab and field settings, to assess user experience well how applications are easy to use.
- QoS testing that evaluates performance, scalability, and availability of mobile services.
- Wireless connectivity testing, such as Satoh's ''flying emulator,'' for mobile network reliability.

- Automation frameworks that attempt to integrate tools and reduce reliance on manual testing.

Tools and Practices

Tools are discussed in this paper is

- Appium
- selendroid
- Calabash
- MonkeyRunner
- and Selenium WebDriver

These tools differ in whether they support native apps, web apps, or both, and they operate through emulation or physical devices.

Challenges

The authors emphasize persistent challenges that still resonate today:

- High costs of setting up realistic test environments across many devices and platforms.
- Lack of standardization in automation frameworks, scripting languages, and protocols.
- Scalability issues, since most tools cannot support large-scale concurrent testing.
- Need for reusable environments, capable of adapting to rapid device and OS updates.

5.2 Literature Review on Security Testing for Mobile Applications Using AI and ML Algorithms

Introduction

Mobile applications are now very important to digital life, banking, healthcare, e-commerce,education, and social platforms. However, mobile application widespread adoption makes them attractive targets for cyberattacks such as data accessible , malware, and unauthorized access to the mobile applications. Traditional security testing methods static and dynamic analysis have some The solution of these is AI and ML algorithms are increasingly applied in mobile application security testing, offering automation, adaptability, and real-time threat detection

Key Contributions

- Paper showing that AI and ML-based models best perform from traditional testing methods in vulnerability detection, efficiency, and false-positive reduction.
- Paper Showed improvements across automated code analysis, anomaly detection, behavioral analysis, and penetration testing.
- Presenting empirical results (e.g., 92
- Highlighting how continuous learning in ML helps keep pace with evolving cyber threats.

Tools and Practices

The Paper showed multiple AI/ML techniques and tools for mobile security testing

- Automated Code Analysis: Uses supervised learning on source code repositories and vulnerability databases to find out insecure patterns.
- Anomaly Detection: Employs unsupervised learning to identify variations in user behavior (e.g., login patterns, abnormal data transfers).
- Behavioral Analysis: Builds user profiles to notice insider threats or sophisticated attacks on not matching known signatures.
- AI-Driven Penetration Testing: Simulates complex attack scenarios with higher accuracy and reduced time compared to human testers.
- Continuous Security Assessment: AI/ML models adapt to updates in mobile applications and growing attack vectors, keeping defenses current.

Challenges

Chanllenges in this paper to integrating AI/ML into mobile app security testing:

- False Positives: Misclassification of legal activity as malicious can disrupt system reliability.
- Data Privacy Concerns: Large datasets used for training may expose sensitive user data.
- Ethical Issues: Responsible handling of behavioral data and compliance with privacy standards are critical.

- Model Training Quality: Requires high-quality, diverse, and representative datasets to ensure robustness.
- Integration into Development Pipelines: Incorporating AI/ML seamlessly into DevSecOps requires standardization and industry collaboration.
- Adaptability vs. Complexity: Continuous retraining is necessary to cope with novel attack methods, which increases complexity

6 Literature review on A systematic mapping study of mobile application testing techniques

Introduction

Mobile applications have become important to modern life, extending beyond entertainment to critical domains such as healthcare, finance, and governance. Unlike traditional web or desktop software, mobile applications facing unique challenges: there are limited device resources, frequently OS upgrades, and changing context inputs. These features demand specialized testing methods to confirm reliability, usability, and security. Recognizing a lack of comprehensive reviews, Zein et al. (2016) organized a systematic mapping study to categorize mobile application testing research and highlight gaps

Key Contribution

The study combined evidence from 79 papers, classifying these papers into five testing domains:

- Usability testing: evaluating effectiveness, efficiency, satisfaction, and user experience.
- Test automation: automating GUI, functional, and regression testing for scalability.
- Context-awareness: addressing challenges from dynamic environments like location, sensors, connectivity.
- Security testing: detecting vulnerabilities, malware, and unsafe inter-app communications.
- General testing: covering broader techniques and frameworks.

Key contributions include: A classification system for mobile testing research. Identifying research gaps, such as insufficient life cycle and lack of empirical validation in real-world settings.

Related Work

earlier reviews were divided:

- Méndez-Porras et al. (2015) in this paper provides a mapping study but limited focus to test automation.
- Muccini et al. (2012) in this Paper conducted an informal review, identifying features of mobile testing and gaps in automation and lifecycle integration.
- Harrison et al. (2013) in this Paper examined usability models, proposing PACMAD for broader usability attributes.
- Gao et al. (2014a, 2014b) in this paper discussed infrastructures and Testing-as-a-Service for scalability and cloud-based validation.

Tools and Practice

The tools used in this paper for emperical studies:

- Automation frameworks (Robotium, AppDoctor, Caiipa, MobiGUITAR).
- Cloud-based and crowdsourced testing platforms (MTaaS, device clouds).
- Usability evaluation tools logging frameworks, eye-tracking, video recording.
- Security tools malware detection using machine learning, vulnerability pattern models.

Challenges

This paper highlights several challenges:

- Contextual complexity: dealing with dynamic inputs such as location, connectivity, and user behavior.
- Platform diversity: testing on multiple operating systems versions and device types.
- Scalability: cost and complexity of large-scale, real-world testing.
- Usability { lack of comprehensive models beyond efficiency and satisfaction.
- Security: fast growth of malware threats and difficulties in testing inter-application communications.

- Lifecycle integration: need for alignments with testing throughout the app development process.